

Backend Frameworks

Ambiente Virtual VENV / Roteiro Inicial do FLASK

Professor: Joseph Donald

🌐 Contatos: www.josephdonald.wordpress.com

📞 (83) 98228-8607

📷 @josephdonald

✉ 030106382@uninassau.edu.br

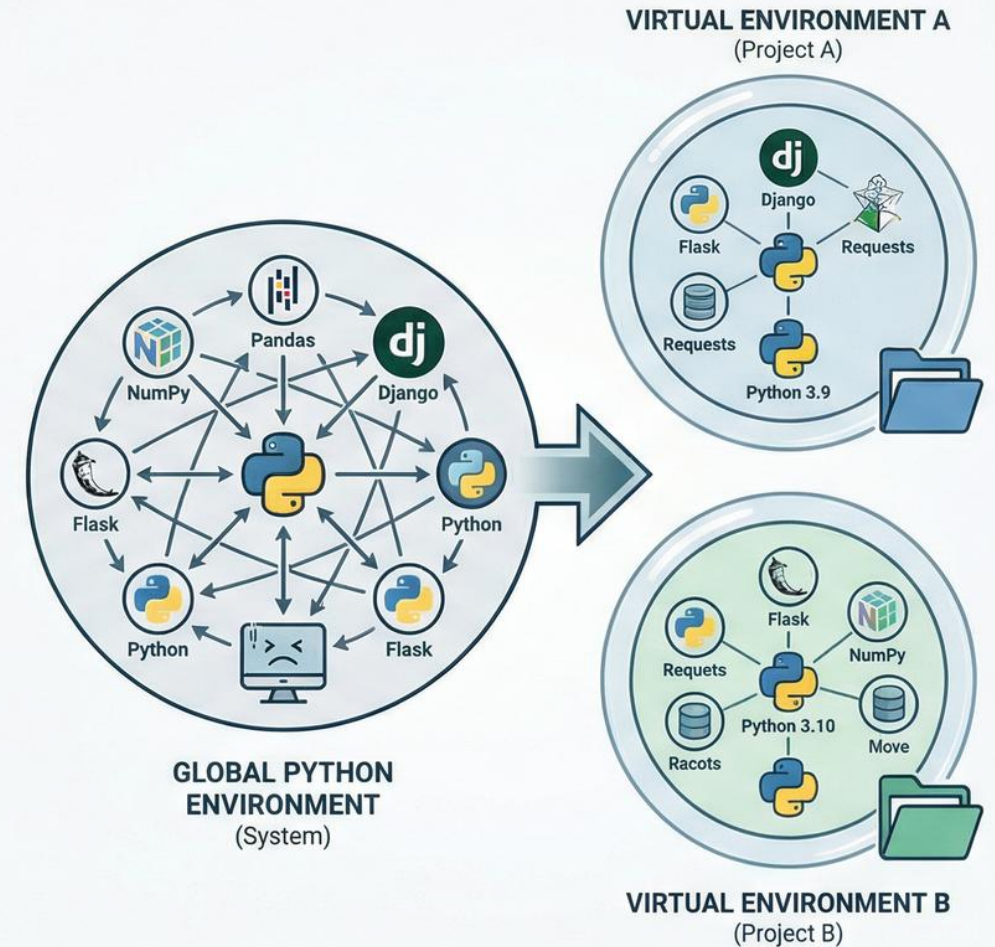
Introdução aos Ambientes Virtuais

✓ **Prática Fundamental:** O uso de ambientes virtuais é uma das práticas mais recomendadas e essenciais no desenvolvimento moderno em Python.

🏗️ **Organização de Dependências:** Eles permitem organizar as bibliotecas e pacotes necessários para diferentes projetos de forma estruturada e independente.

🛡️ **Prevenção de Conflitos:** Ao isolar o ambiente, evita-se conflitos diretos entre bibliotecas instaladas globalmente no sistema operacional do computador.

🐍 **Suporte Nativo:** É um recurso oficial e integrado nativamente à linguagem Python a partir da versão 3.3, dispensando a instalação de ferramentas externas para sua criação básica.



Isolation & Organization: From Global Chaos to Project-Specific Stability

O que é um Ambiente Virtual (venv)?



Definição: Basicamente, trata-se de uma cópia isolada do interpretador Python, configurada para atuar em um espaço restrito.



Diretório Exclusivo: O ambiente virtual possui o seu próprio diretório de bibliotecas, separado da instalação principal do Python no sistema.



Independência de Projetos: Garante que cada projeto em desenvolvimento possa ter suas próprias dependências de forma totalmente autônoma e segura.



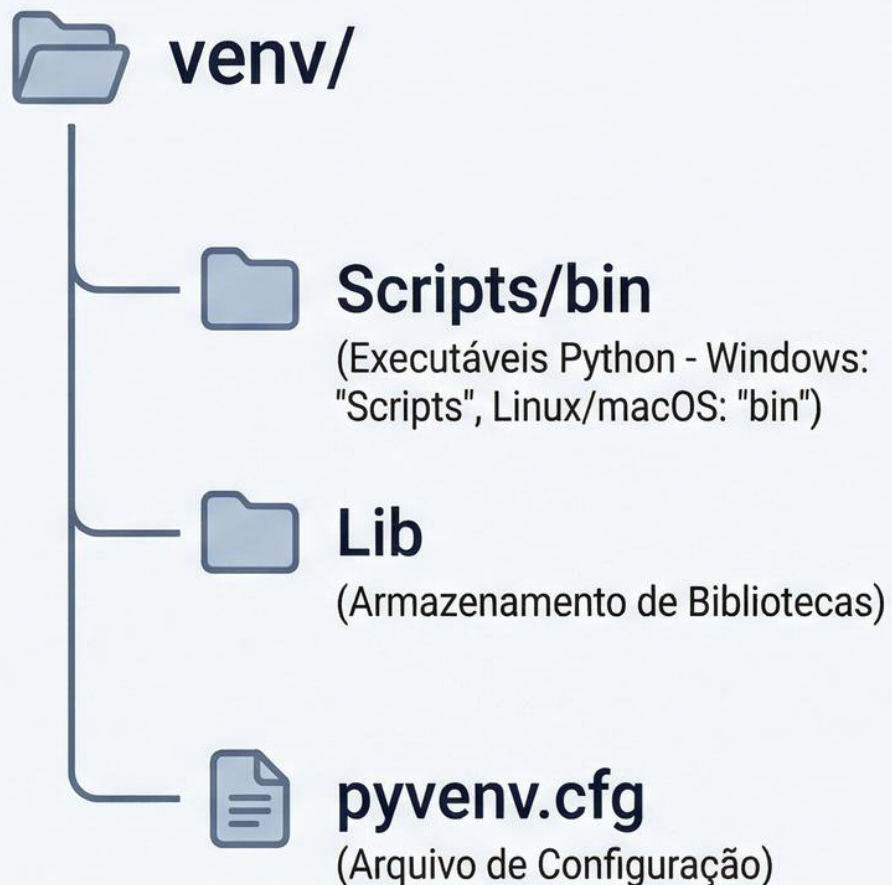
Coexistência de Versões: Permite que múltiplas versões de uma mesma biblioteca (ou até mesmo do framework) coexistam na mesma máquina sem causar falhas no sistema global.

O Problema de Não Utilizar o venv



- **Cenário Prático:** Desenvolvendo dois projetos no mesmo computador com requisitos diferentes.
- **Conflito de Versões:** O Projeto A necessita estritamente do Flask 2.0, enquanto o Projeto B exige o Flask 3.0.
- **O Risco das Instalações Globais:** A atualização para o Projeto B pode quebrar irreversivelmente o Projeto A no ambiente compartilhado.

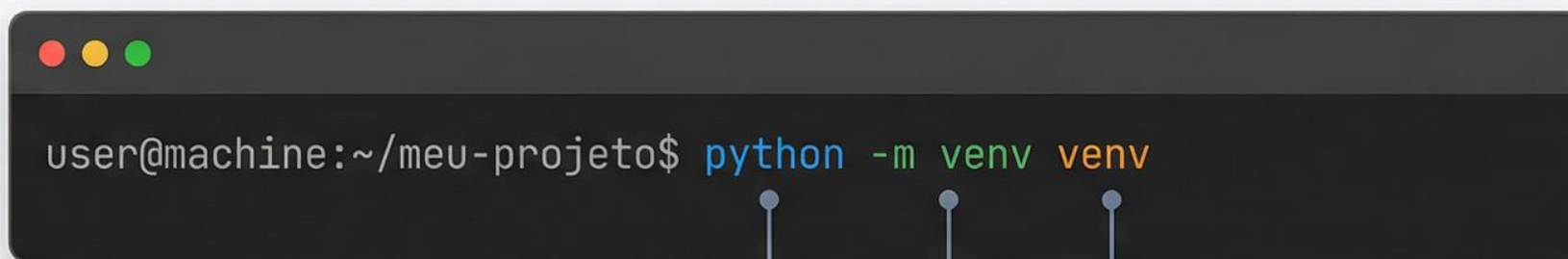
Estrutura de um Ambiente Virtual



- ▶ **Geração do Diretório:** O processo de criação gera uma pasta fisicamente isolada na raiz do seu projeto (geralmente nomeada como "venv/").
- ▶ **Executáveis (Scripts/bin):** Contém as pastas de scripts executáveis do Python. No sistema Windows chama-se "Scripts", enquanto no Linux ou macOS chama-se "bin".
- **Armazenamento de Bibliotecas:** Contém a pasta 'Lib' (Library), que é o local onde as bibliotecas de terceiros instaladas pelo projeto ficam efetivamente armazenadas.
- **Arquivo de Configuração:** Possui o arquivo essencial 'pyvenv.cfg', responsável por guardar as configurações originais e as chaves de isolamento do ambiente.

Como Criar um Ambiente Virtual

Passo Inicial: O primeiro passo é acessar a pasta raiz do seu projeto utilizando a interface de linha de comando (terminal).



```
user@machine:~/meu-projeto$ python -m venv venv
```

Comando de Criação: Para criar o ambiente, execute o comando padrão:



Chamada Primária

A primeira palavra 'python' faz a chamada primária ao interpretador global do sistema operacional.



Execução de Módulo

O parâmetro '-m venv' instrui o Python a executar o módulo interno responsável por gerar ambientes virtuais.

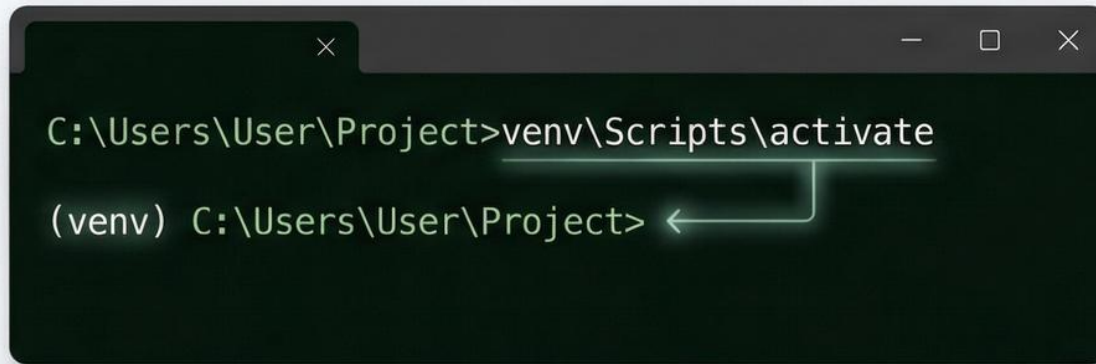


Nomenclatura

O último 'venv' da instrução apenas define o nome da pasta a ser criada (que poderia ser 'env', '.venv', etc., embora 'venv' seja a convenção).

Como Ativar o Ambiente Virtual

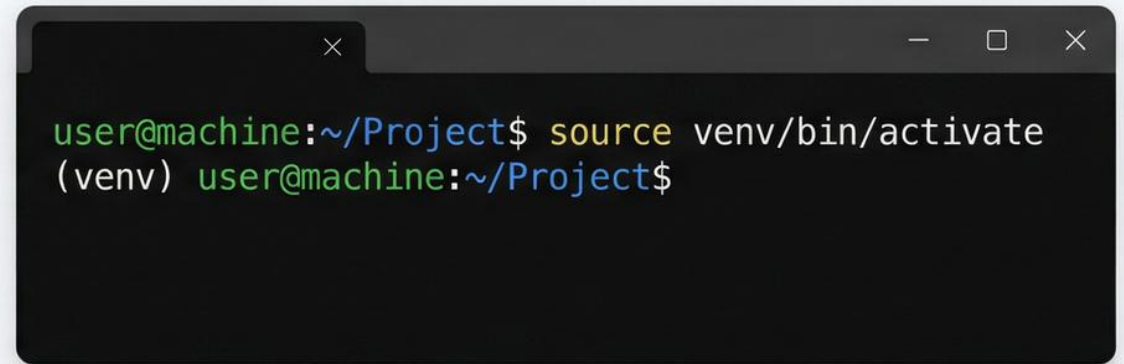
Windows



```
C:\Users\User\Project>venv\Scripts\activate  
(venv) C:\Users\User\Project>
```

A terminal window with a dark background. The first line shows the command `venv\Scripts\activate` being executed from the directory `C:\Users\User\Project`. The second line shows the prompt changing to `(venv) C:\Users\User\Project>`, indicating the virtual environment is active. A green arrow points from the underlined `activate` command to the `(venv)` prefix in the subsequent prompt.

Linux / macOS



```
user@machine:~/Project$ source venv/bin/activate  
(venv) user@machine:~/Project$
```

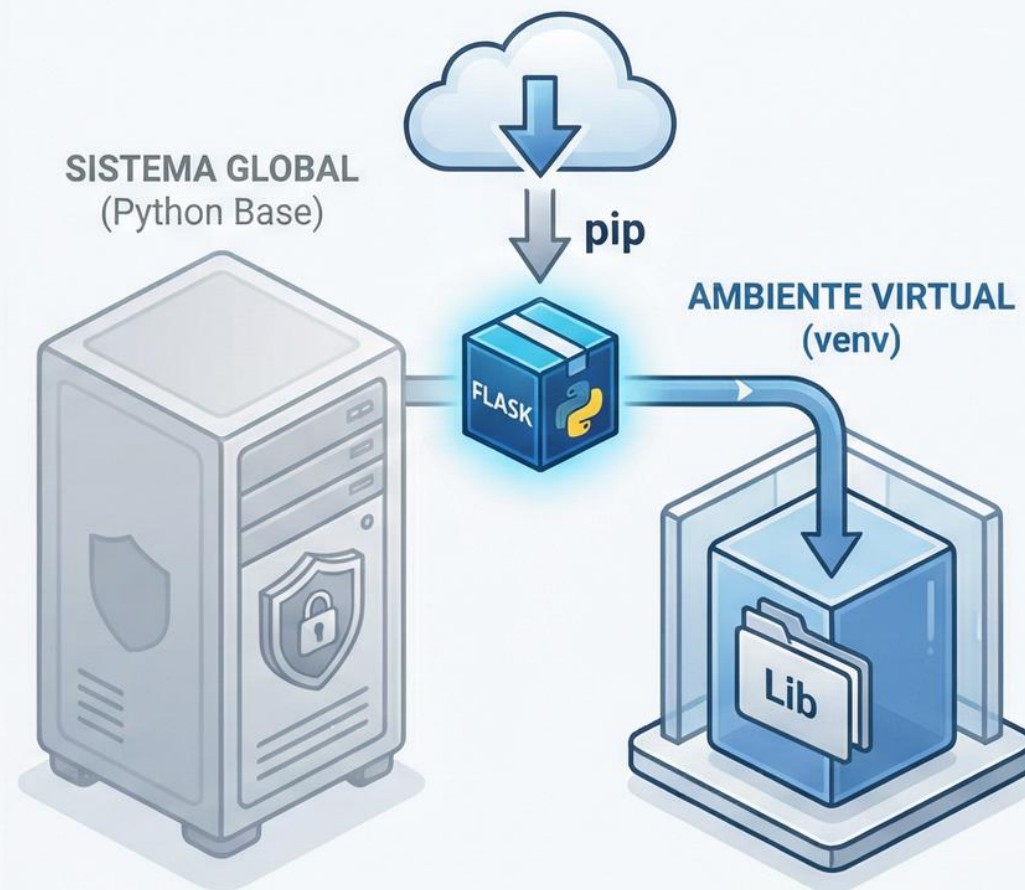
A terminal window with a dark background. The first line shows the command `source venv/bin/activate` being executed from the directory `~/Project`. The second line shows the prompt changing to `(venv) user@machine:~/Project$`, indicating the virtual environment is active.

Feedback Visual e Significado

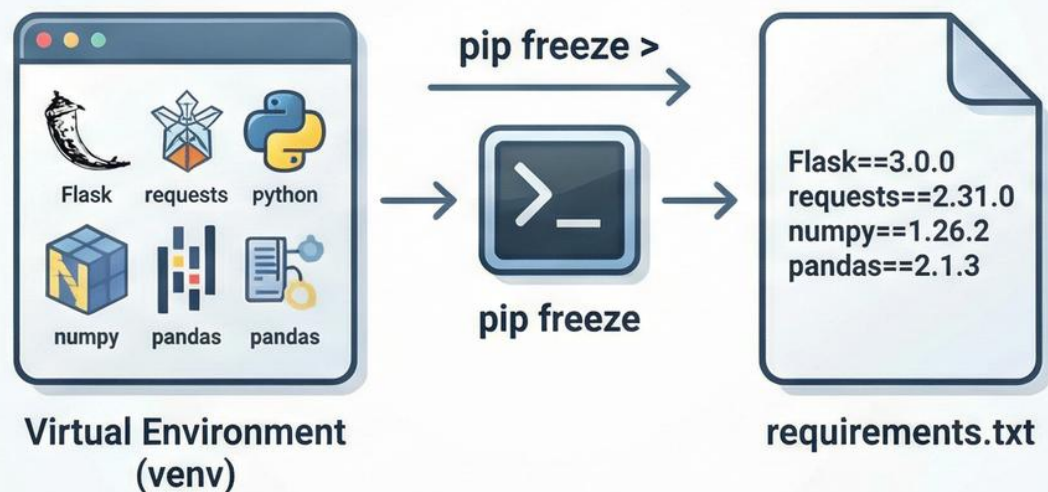
- ✓ Imediatamente após a execução, o terminal passará a exibir o prefixo '(venv)' antes do caminho do diretório atual.
- i Esse indicador visual é a garantia do sistema de que o ambiente isolado está pronto e ativo para o uso seguro de comandos e instalações.

Instalando Bibliotecas no venv

-  **Uso do PIP:** Com o ambiente virtual já devidamente ativo, você deve utilizar o 'pip', que é o gerenciador de pacotes padrão do Python.
-  **Comando de Instalação:** O processo de instalação ocorre através do comando de exemplo: 'pip install flask'.
-  **Garantia de Isolamento:** Toda instalação ou atualização de biblioteca ocorrerá de forma restrita e exclusiva apenas dentro da pasta 'Lib' do seu ambiente virtual.
-  **Ambiente Global Protegido:** Dessa forma, o interpretador Python global do seu computador jamais será modificado, sobreposto ou "poluído" com bibliotecas de testes.

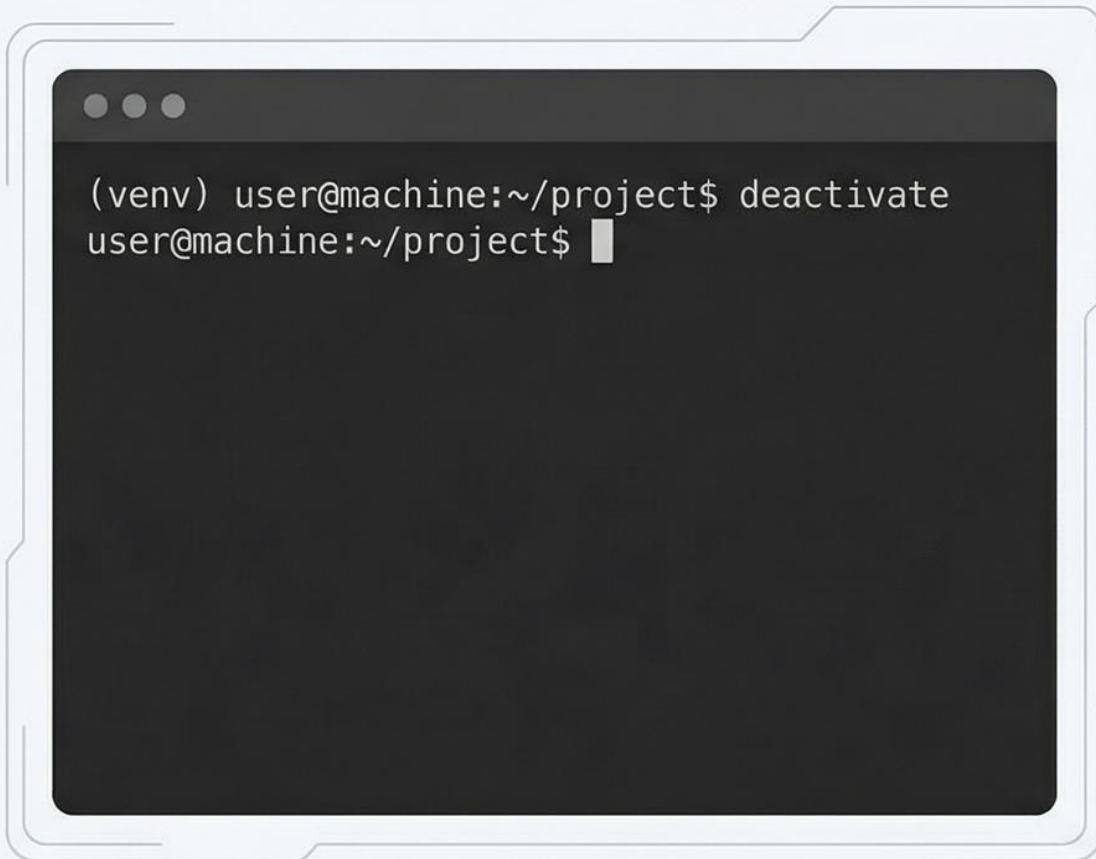


Registrando Dependências: requirements.txt



- **Propósito do Arquivo:** Este arquivo de texto simples serve como uma "receita", registrando todas as dependências necessárias de um projeto.
- **Geração Automática:** Para criar ou atualizar o arquivo, execute o comando no terminal: "pip freeze > requirements.txt".
- **Detalhamento de Versões:** O comando varre o venv e cria uma lista exata mapeando cada biblioteca instalada para sua versão específica (ex: Flask==3.0.0).
- **Compartilhamento e Reprodutibilidade:** Facilita enormemente o trabalho em equipe, permitindo que outro desenvolvedor (ou servidor) baixe o código e instale tudo rapidamente com: 'pip install -r requirements.txt'.

Desativando o Ambiente Virtual

A terminal window with a dark background and light text. The prompt is '(venv) user@machine:~/project\$'. The command 'deactivate' has been entered, and the prompt has changed to 'user@machine:~/project\$'.

```
(venv) user@machine:~/project$ deactivate
user@machine:~/project$
```

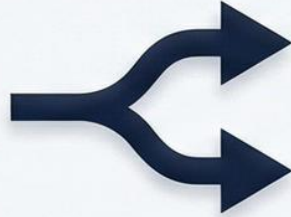
- **Finalização Segura:** Quando você finalizar as tarefas de desenvolvimento no projeto atual, é uma boa prática desativar o ambiente.
- **Comando Simples:** Em qualquer sistema operacional, basta digitar o comando simples '**deactivate**' no seu terminal.
- **Feedback Visual de Saída:** Imediatamente após confirmar o comando, o prefixo '(venv)' desaparecerá da sua linha de comando.
- **Retorno ao Padrão:** A partir desse momento, o sistema operacional voltará a utilizar o interpretador Python global padrão para a execução de qualquer script.

Vantagens do Uso de venv



Isolamento Completo

Permite o isolamento estrito de todas as dependências por projeto individual.



Controle de Versões

Viabiliza um controle rigoroso de versões de pacotes em múltiplos projetos simultâneos.



Reprodutibilidade Garantida

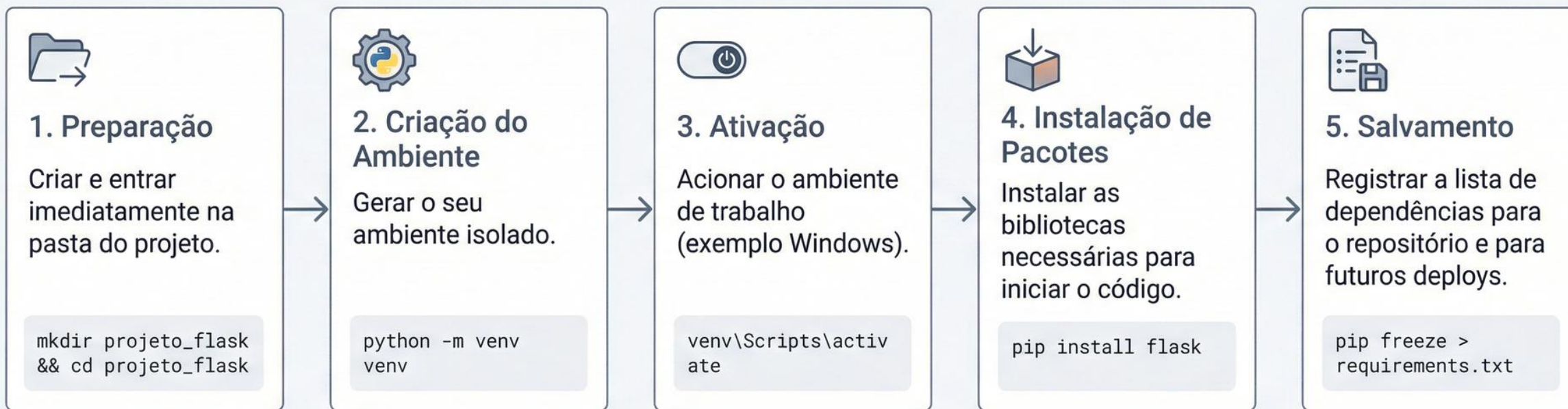
A utilização atrelada ao arquivo 'requirements.txt' garante que a aplicação funcionará em qualquer computador ou servidor da mesma maneira.



Padrão de Mercado

Trata-se de uma prática profissional fortemente recomendada na indústria, indispensável em ecossistemas de frameworks web, análise e ciência de dados.

Exemplo de Fluxo Completo de Trabalho



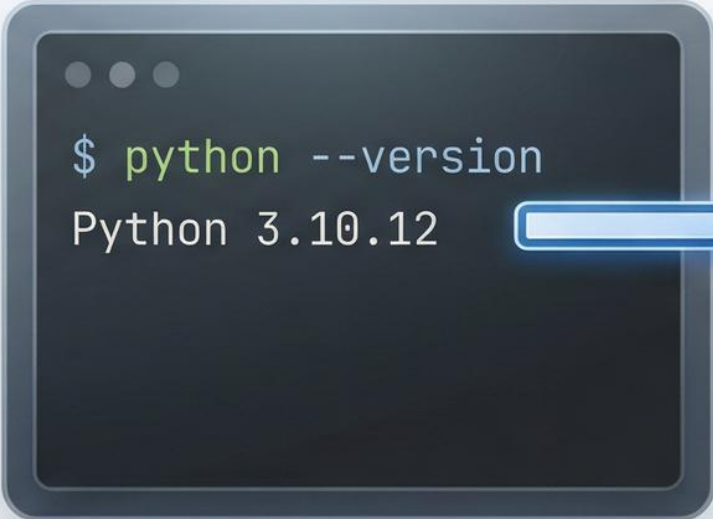
O que é o Flask?



- **Definição Central:** O Flask é conhecido como um "microframework" web focado e otimizado para o desenvolvimento veloz de aplicações em Python.
- **Origem e Manutenção:** Foi originalmente criado por Armin Ronacher e atualmente é mantido ativamente pela comunidade open-source Pallets Projects.
- **Foco no Essencial:** Diferente de frameworks monolíticos (como Django), o Flask fornece apenas os componentes primordiais e essenciais para se criar aplicações web (roteamento, respostas, etc.).
- **Alta Flexibilidade:** Sua arquitetura enxuta permite ao desenvolvedor total liberdade para escolher e adicionar bibliotecas complementares (ex: bancos de dados, autenticação) apenas quando estritamente necessário.

Preparando o Ambiente para o Flask

- **Pré-requisito Obrigatório:** Antes de iniciar qualquer codificação, é mandatório confirmar que o Python está corretamente instalado no computador.
- **Verificação de Versão:** Utilize o terminal para checar a instalação com o comando: 'python --version' (ou 'python3 --version' em sistemas Unix).
- **Feedback do Sistema:** O terminal deverá exibir em texto plano a versão atual do Python reconhecida pelo sistema operacional.
- **Boas Práticas Imediatas:** Antes de aplicar o "pip install flask", crie e ative **obrigatoriamente** um novo ambiente virtual, adotando a postura profissional discutida anteriormente.



```
$ python --version  
Python 3.10.12
```

A terminal window with a dark background and light text. The command `$ python --version` is entered, and the output `Python 3.10.12` is displayed. A large blue arrow points from the terminal to the right.



Ambiente Virtual (venv)
Isolado & Profissional

Criando a Primeira Aplicação Flask

Instalação do Framework

Certifique-se de que o venv está ativo e instale o pacote via gerenciador: 'pip install flask'.

Arquivo Principal

Crie um arquivo Python central para iniciar a construção do seu código. Por convenção na comunidade, ele geralmente é chamado de 'app.py'.

Importação Inicial

Abra o arquivo no editor de código e faça a importação da classe matriz: 'from flask import Flask'.

Instanciação da Aplicação

Logo abaixo, crie a instância principal que representará a aplicação web em si, declarando: 'app = Flask(__name__)'.

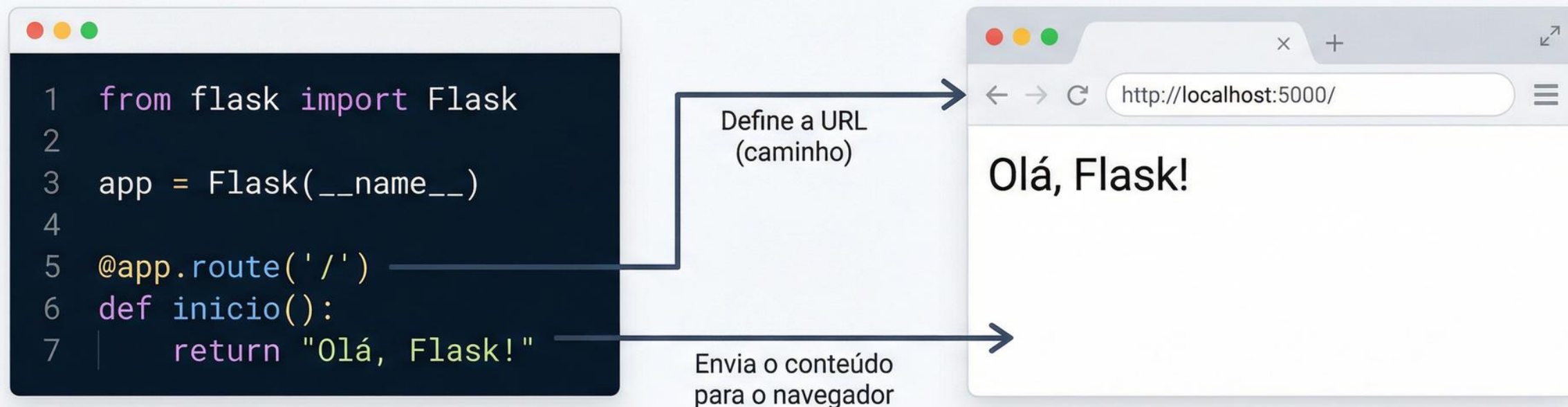


```
$ pip install flask
```

app.py

```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def hello_world():
7      return 'Hello, World!'
```

Entendendo as Rotas no Flask



Conceito de Rota:

- As rotas são a espinha dorsal da aplicação; elas definem diretamente as URLs (caminhos) que o usuário poderá acessar.

Uso de Decoradores:

- Utilize o decorador '@app.route('/')' imediatamente antes de uma função para mapeá-la, definindo assim a rota da página inicial da sua aplicação web.

Associação Lógica:

- Este decorador deve ser intimamente associado a uma função Python padrão, como por exemplo: 'def inicio():'.

Ação de Retorno:

- Essa função associada é obrigada a possuir um comando 'return', que enviará a resposta (texto ou HTML) com o conteúdo a ser exibido na tela do navegador do cliente.

Executando e Acessando a Aplicação

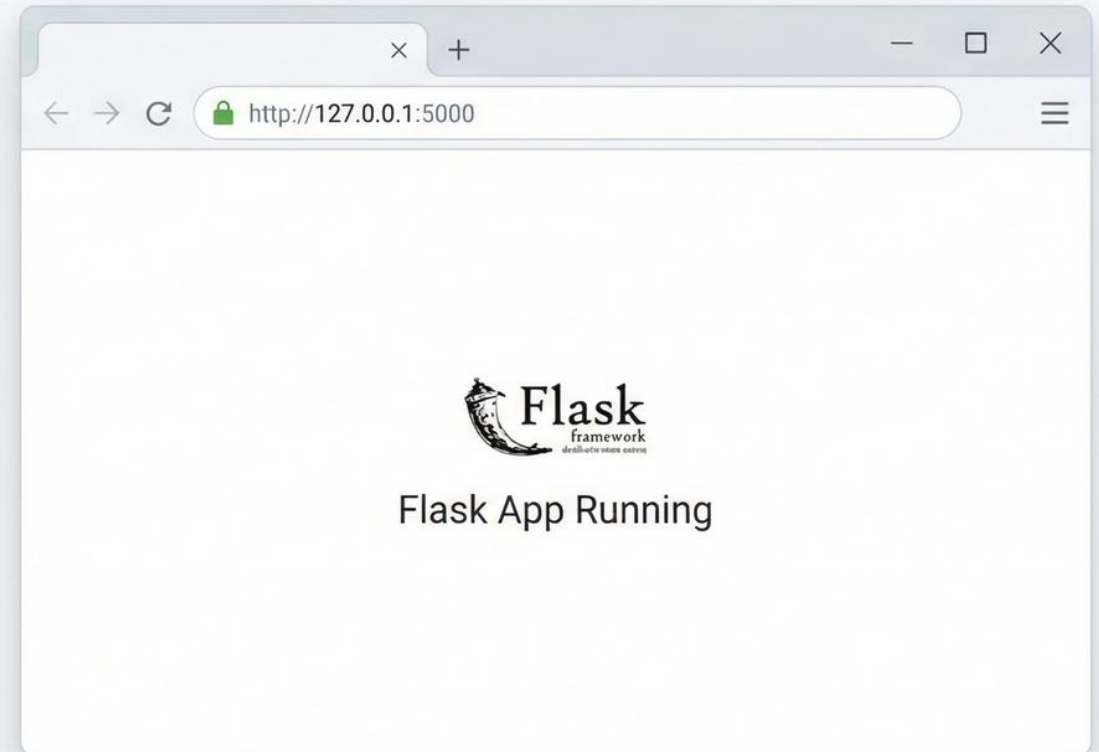
```
1 if __name__ == "__main__":  
2     app.run(debug=True)  
3  
$ python app.py
```

Código de Execução: Para manter o servidor rodando, adicione o seguinte trecho condicional no final do arquivo.

Acionamento via Terminal: Salve o arquivo e execute a aplicação diretamente no terminal através do comando.

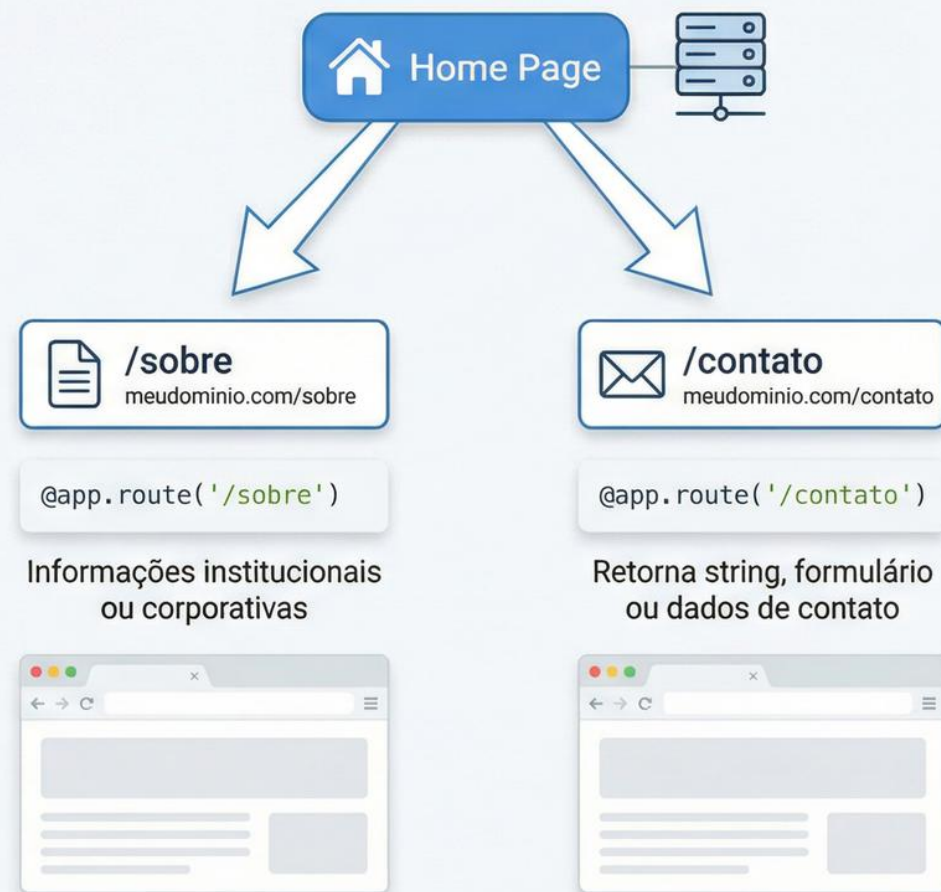
Vantagens do Modo Debug:

A utilização do parâmetro 'debug=True' permite que o servidor reinicie e atualize automaticamente a cada salvamento do código e exibe logs detalhados, o que facilita enormemente encontrar e resolver erros.



Criando Novas Rotas Dinâmicas

-  **Multiplicidade de Páginas:** Lembre-se que uma única aplicação web pode e deve abrigar diversas páginas e endpoints diferentes, criando uma navegação rica.
-  **Exemplo de Rota Estática de Informação:** Utilize o decorador '@app.route('/sobre')' acionando uma função correspondente para detalhar informações institucionais ou corporativas.
-  **Exemplo de Rota Estática de Contato:** Crie um endpoint com '@app.route('/contato')' retornando uma string, formulário ou dados de contato específicos.
-  **Independência de Conteúdo:** Ao acessar os diferentes endereços (ex: meudominio.com/sobre ou meudominio.com/contato), cada URL será interceptada por sua própria função, que exibirá um conteúdo web estritamente focado e independente.



Utilizando Templates HTML



Mecanismo Jinja2

O Flask não retorna apenas texto simples. Ele integra códigos HTML dinâmicos utilizando um poderoso e seguro mecanismo de templates chamado 'Jinja2'.



Importação Essencial

Para utilizar este recurso vital, importe a função específica 'render_template' logo ao lado do pacote principal 'Flask' no cabeçalho do código.



Padronização de Diretório

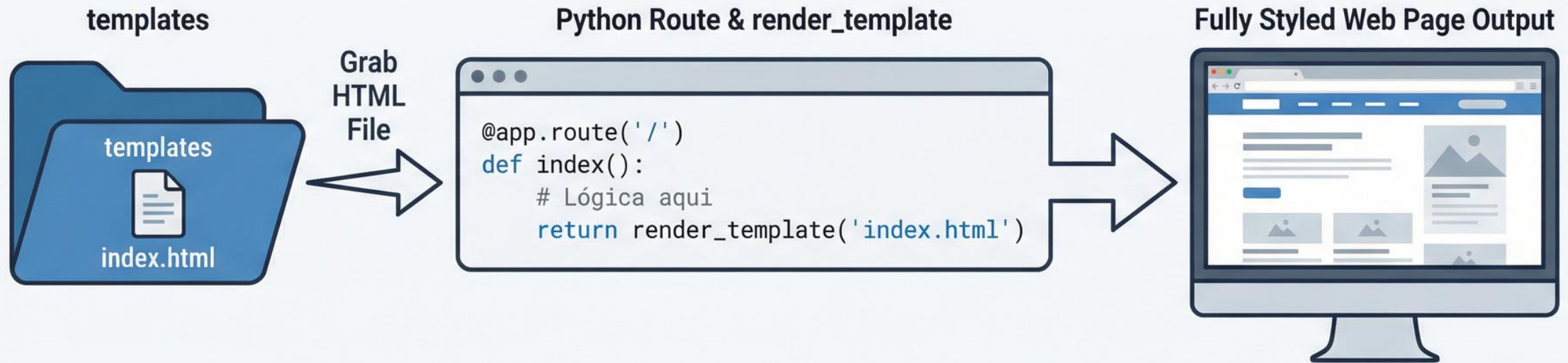
É obrigatório criar uma pasta interna com o nome exato de 'templates' (tudo em minúsculo), localizada estritamente na raiz do seu projeto ao lado do arquivo 'app.py'.



Armazenamento de Visuais

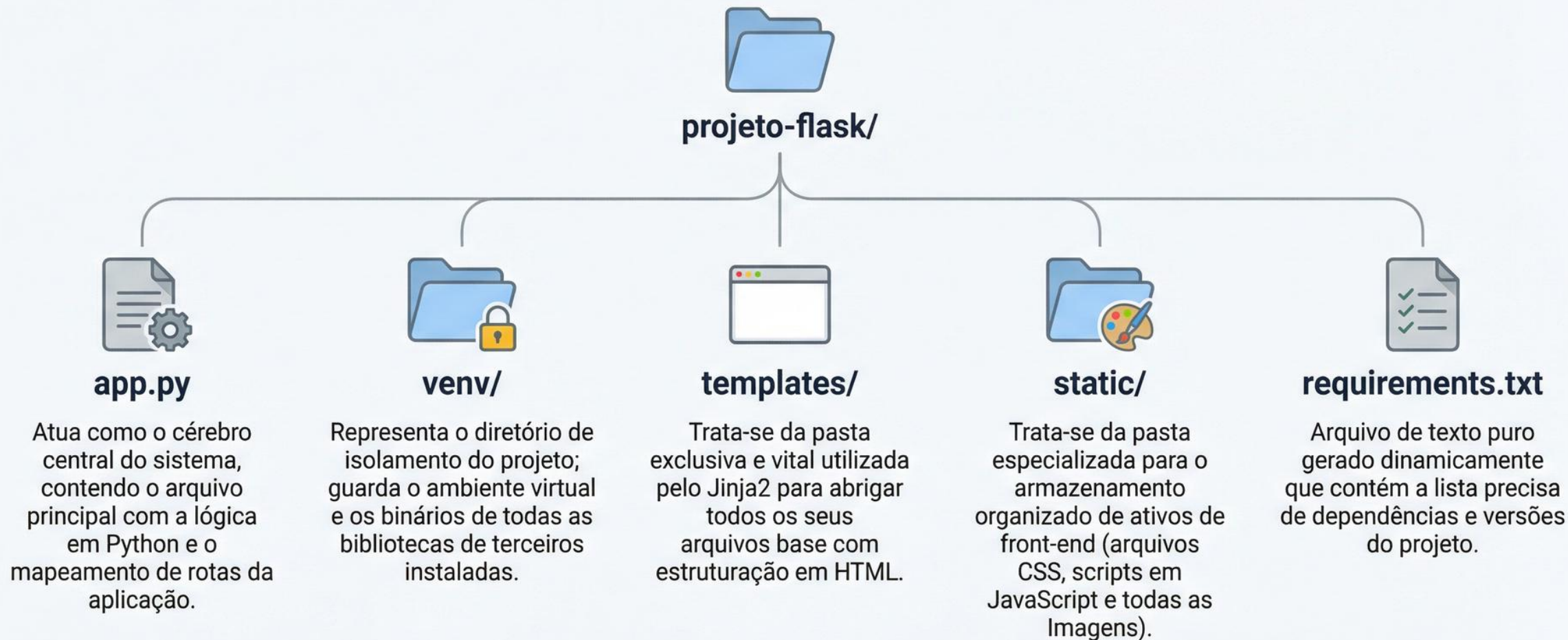
A partir desse momento, salve todos os seus arquivos de estruturação web (como por exemplo o arquivo padrão 'index.html') sempre dentro desta pasta reservada.

Renderizando Páginas Completas HTML

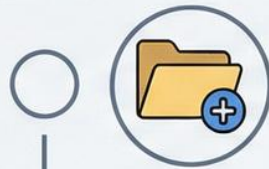


- **Ajuste da Função de Rota:** Substitua o seu retorno original em formato de texto cru na função da rota por uma chamada formal ao arquivo visual (template).
- **Sintaxe da Renderização:** Aplique o código de exemplo no 'return' de sua função: `'return render_template('index.html')'`.
- **Busca Automatizada:** Graças à arquitetura do framework, o Flask buscará **invisivelmente** e de forma **automática** o arquivo exato que você solicitou dentro da pasta padronizada **'templates'**.
- **Interfaces Modernas e Avançadas:** Essa abordagem de separação entre Lógica (Python) e Visão (HTML) permite criar interfaces web extremamente ricas, bonitas, visualmente estruturadas e escaláveis para aplicações do mundo real.

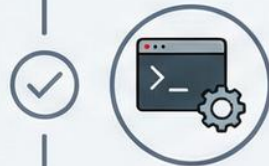
Estrutura Básica de um Projeto Flask Completo



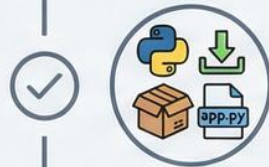
Desafio 1: Configuração do Ambiente de Desenvolvimento



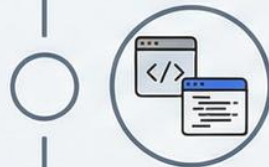
Passo 1 (Diretório): Crie, através do terminal ou de seu explorador de arquivos, uma nova pasta denominada "meu_primeiro_flask".



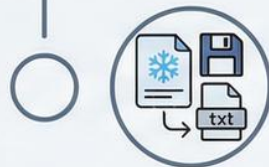
Passo 2 (Isolamento): Acesse o interior desta pasta pelo terminal, instancie um novo ambiente virtual ("venv") e realize a ativação imediata do mesmo.



Passo 3 (Pacote e Arquivo): Execute o gerenciador "pip" para instalar o framework web Flask e crie, na raiz do projeto, um arquivo base nomeado como "app.py".



Passo 4 (Codificação): Programe uma rota principal de acesso padrão ("/") que possua como retorno estático a frase exata: "Sistema Flask funcionando corretamente!".



Passo 5 (Fechamento): Ao final de todo o processo lógico e de sucesso, aplique o "pip freeze" para gerar e salvar o histórico final em um arquivo "requirements.txt".

Desafio 2: Múltiplas Rotas Dinâmicas (Parte 1)

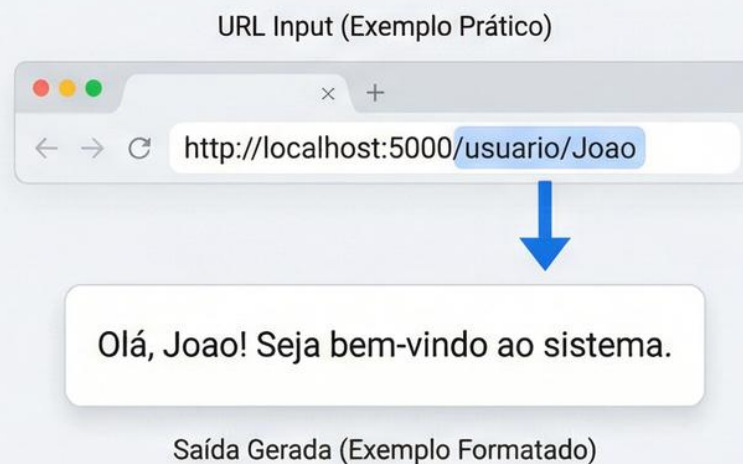
Implementação de Rotas

- **Rota Principal (/):**

Crie uma rota raiz cujo objetivo será exibir unicamente a string de boas-vindas na tela: "Bem-vindo ao sistema de testes Flask".

- **Rota de Saudação (/usuario/<nome>):**

Programa uma rota avançada e dinâmica, que utilize nativamente a passagem dinâmica de parâmetros contida diretamente no corpo da URL de acesso.



Lógica e Formatação

Lógica de Captura:

Desenvolva a função Python associada à rota garantindo que ela consiga capturar internamente o parâmetro "nome" que o usuário final digitar **livremente na barra** de endereços do seu navegador web.

Formatação de Resposta:

Para testar a funcionalidade da captura e resposta, o sistema deverá **retornar diretamente na tela** uma mensagem formatada e personalizada. Exemplo prático: "Olá, Joao! Seja bem-vindo ao sistema.".

Desafio 2: Múltiplas Rotas Dinâmicas (Parte 2)



Implementação da Rota de Soma (`/soma/<int:a>/<int:b>`)

- Desenvolva um endpoint que obrigatoriamente aguarde a recepção de dois parâmetros tipados como números inteiros ('a' e 'b') através da própria URL.
- Operação Matemática: Com os parâmetros em mãos, efetue a rotina de processamento de adição ($a + b$) através da lógica em Python e entregue como retorno a mensagem padronizada "A soma de $a + b = X$ " (com X sendo o cálculo realizado).

Implementação da Rota de Paridade (`/paridade/<int:numero>`)

- Construa uma terceira rota especializada em avaliar e receber apenas um número do tipo inteiro na entrada da requisição de URL.
- Processamento Condicional: Avalie através do operador matemático '%' a natureza de divisão do número capturado e elabore o código para verificar sua paridade de forma rígida, exibindo na tela como finalidade a frase 'O número X é PAR' ou, alternativamente, 'ÍMPAR'.

Conclusão e Encerramento



O domínio do uso de ambientes virtuais (venv) é vital, sendo o marco inicial e o primeiro passo sólido para um desenvolvimento profissional em Python de forma segura e padronizada.



A adoção de tecnologias modernas como o microframework Flask é incrivelmente produtiva, viabilizando e facilitando a ideação e criação rápida e descomplicada de sistemas web robustos.



A habilidade em compreender e utilizar adequadamente a sintaxe de rotas dinâmicas, atreladas ao poder de exibição do Jinja2 e do HTML, eleva o código para a criação de sistemas web completos e atraentes no mercado de tecnologia.

Obrigado a todos pela atenção durante a apresentação de hoje! Estarei deixando este espaço final totalmente aberto para eventuais esclarecimentos de dúvidas e colaborações construtivas.